

Defining Data Types

CS 350

Dr. Joseph Eremondi

January 22, 2025

Warmup: Pairs

Algebraic Data Types

Warmup: Pairs

Pairs: “AND” for types

- (Number * Boolean)

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean
 - Why is it infix? Who knows

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean
 - Why is it infix? Who knows
 - Maybe to help distinguish it from multiplication

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean
 - Why is it infix? Who knows
 - Maybe to help distinguish it from multiplication
- Projections

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean
 - Why is it infix? Who knows
 - Maybe to help distinguish it from multiplication
- Projections
 - Get the data from the pairs

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean
 - Why is it infix? Who knows
 - Maybe to help distinguish it from multiplication
- Projections
 - Get the data from the pairs

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean
 - Why is it infix? Who knows
 - Maybe to help distinguish it from multiplication
- Projections
 - Get the data from the pairs

```
(define myPair : (Number * Boolean)
  (pair 2 #t))
(fst myPair) ;; Number
(snd myPair) ;; Boolean
```

Pairs: “AND” for types

- (Number * Boolean)
 - Cartesian product
 - “AND” for types
 - A value of this type contains a Number AND a Boolean
 - Why is it infix? Who knows
 - Maybe to help distinguish it from multiplication
- Projections
 - Get the data from the pairs

```
(define myPair : (Number * Boolean)
  (pair 2 #t))
(fst myPair) ;;Number
(snd myPair) ;;Boolean
```

```
2
#t
```

- For any types 'a and 'b there is a type ('a * 'b)

Pairs in General

- For any types 'a and 'b there is a type ('a * 'b)
- Build with pair : ('a 'b -> ('a * 'b))

Pairs in General

- For any types `'a` and `'b` there is a type `('a * 'b)`
- Build with `pair` : `('a 'b -> ('a * 'b))`
 - Takes one argument of each type, produces the pair

Pairs in General

- For any types 'a and 'b there is a type ('a * 'b)
- Build with pair : ('a 'b -> ('a * 'b))
 - Takes one argument of each type, produces the pair
- Projections

Pairs in General

- For any types 'a and 'b there is a type ('a * 'b)
- Build with pair : ('a 'b -> ('a * 'b))
 - Takes one argument of each type, produces the pair
- Projections
 - fst : (('a * 'b) -> 'a)

Pairs in General

- For any types `'a` and `'b` there is a type `('a * 'b)`
- Build with `pair` : `('a 'b -> ('a * 'b))`
 - Takes one argument of each type, produces the pair
- Projections
 - `fst` : `(( 'a * 'b) -> 'a)`
 - `snd` : `(( 'a * 'b) -> 'b)`

Pairs in General

- For any types `'a` and `'b` there is a type `('a * 'b)`
- Build with `pair` : `('a 'b -> ('a * 'b))`
 - Takes one argument of each type, produces the pair
- Projections
 - `fst` : `(( 'a * 'b) -> 'a)`
 - `snd` : `(( 'a * 'b) -> 'b)`
- Used for functions that return multiple values

Pairs in General

- For any types `'a` and `'b` there is a type `('a * 'b)`
- Build with `pair` : `('a 'b -> ('a * 'b))`
 - Takes one argument of each type, produces the pair
- Projections
 - `fst` : `(( 'a * 'b) -> 'a)`
 - `snd` : `(( 'a * 'b) -> 'b)`
- Used for functions that return multiple values
 - Or to pass two values as one to a function

Pairs as Anonymous Records

- Pairs similar to two-field structs in C++

Pairs as Anonymous Records

- Pairs similar to two-field structs in C++
 - Main difference: no need to make a new name for pairs

Pairs as Anonymous Records

- Pairs similar to two-field structs in C++
 - Main difference: no need to make a new name for pairs
 - Can write generic code that works for all pairs

Pairs as Anonymous Records

- Pairs similar to two-field structs in C++
 - Main difference: no need to make a new name for pairs
 - Can write generic code that works for all pairs

Pairs as Anonymous Records

- Pairs similar to two-field structs in C++
 - Main difference: no need to make a new name for pairs
 - Can write generic code that works for all pairs

```
(define (flip [pr : ('a * 'b)]) : ('b * 'a)  
  (pair (snd pr) (fst pr)))  
(flip (pair 3 4))
```

- Python has pairs/tuples as a core datatype

Pairs as Anonymous Records

- Pairs similar to two-field structs in C++
 - Main difference: no need to make a new name for pairs
 - Can write generic code that works for all pairs

```
(define (flip [pr : ('a * 'b)]) : ('b * 'a)  
  (pair (snd pr) (fst pr)))  
(flip (pair 3 4))
```

- Python has pairs/tuples as a core datatype
 - Tuple: equivalent of pair, but with n elements for some n

Pairs as Anonymous Records

- Pairs similar to two-field structs in C++
 - Main difference: no need to make a new name for pairs
 - Can write generic code that works for all pairs

```
(define (flip [pr : ('a * 'b)]) : ('b * 'a)  
  (pair (snd pr) (fst pr)))  
(flip (pair 3 4))
```

- Python has pairs/tuples as a core datatype
 - Tuple: equivalent of pair, but with n elements for some n
 - ('a * b' * 'c * d' ...)

Defining the Pair type

- Pairs are built in, but we could define them like this:

Defining the Pair type

- Pairs are built in, but we could define them like this:

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- Pair is the name of the type

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- Pair is the name of the type
- 'a and 'b are the type parameters

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- Pair is the name of the type
- 'a and 'b are the type parameters
 - e.g. the types of things in the pair

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- `Pair` is the name of the type
- `'a` and `'b` are the type parameters
 - e.g. the types of things in the pair
- There is one constructor, called `pair`

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- Pair is the name of the type
- 'a and 'b are the type parameters
 - e.g. the types of things in the pair
- There is one constructor, called pair
 - type name uppercase, constructor lowercase

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- Pair is the name of the type
- 'a and 'b are the type parameters
 - e.g. the types of things in the pair
- There is one constructor, called pair
 - type name uppercase, constructor lowercase
- It has two fields

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- `Pair` is the name of the type
- `'a` and `'b` are the type parameters
 - e.g. the types of things in the pair
- There is one constructor, called `pair`
 - type name uppercase, constructor lowercase
- It has two fields
 - `fst` and `snd` are just names we choose for the fields

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- `Pair` is the name of the type
- `'a` and `'b` are the type parameters
 - e.g. the types of things in the pair
- There is one constructor, called `pair`
 - type name uppercase, constructor lowercase
- It has two fields
 - `fst` and `snd` are just names we choose for the fields
- Fields have types `'a` and `'b`

Defining the Pair type

- Pairs are built in, but we could define them like this:

```
(define-type (Pair 'a 'b)  
  [pair [elem1 : 'a] [elem2 : 'b]])
```

- `Pair` is the name of the type
- `'a` and `'b` are the type parameters
 - e.g. the types of things in the pair
- There is one constructor, called `pair`
 - type name uppercase, constructor lowercase
- It has two fields
 - `fst` and `snd` are just names we choose for the fields
- Fields have types `'a` and `'b`
 - e.g. the types of the things in the pair

Example: Odds and Evens

Example: Odds and Evens

```
(define (splitOddsEvens [nums : (Listof Number)])  
  : ((Listof Number) * (Listof Number))  
  (type-case (Listof Number) nums  
    ;; Empty list has no odds or evens  
    [(empty)  
     (pair '() '())]  
    [(cons h t)  
     ;; Recursion splits tail into odds/evens  
     ;; use fst and snd to get these two lists  
     (let* ([splitTail (splitOddsEvens t)]  
             [tailOdds (fst splitTail)]  
             [tailEvens (snd splitTail)])  
       ;; Now check if the head is odd or even  
       ;; and add head to correct list  
       (if (= 0 (modulo h 2))  
         ;; If even, add head to the second list  
         (pair tailOdds (cons h tailEvens))  
         ;; If odd, add head to the first list  
         (pair (cons h tailOdds) tailEvens))))))
```

Example: Odds and Evens (ctd)

Example: Odds and Evens (ctd)

```
(splitOddsEvens (list 1 2 3 4 5 7 11 2 4 7))
```

Example: Odds and Evens (ctd)

```
(splitOddsEvens (list 1 2 3 4 5 7 11 2 4 7))
```

```
(values '(1 3 5 7 11 7) '(2 4 2 4))
```

Algebraic Data Types

“OR” for types

- Pairs gave us “AND”

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean
 - Want a tag so we can check which one it is

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean
 - Want a tag so we can check which one it is
 - Called a “constructor”

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean
 - Want a tag so we can check which one it is
 - Called a “constructor”
 - not the same as Java/OOP constructor

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean
 - Want a tag so we can check which one it is
 - Called a “constructor”
 - not the same as Java/OOP constructor
- Saw some examples like this already

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean
 - Want a tag so we can check which one it is
 - Called a “constructor”
 - not the same as Java/OOP constructor
- Saw some examples like this already
 - A number is zero OR one plus another number

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean
 - Want a tag so we can check which one it is
 - Called a “constructor”
 - not the same as Java/OOP constructor
- Saw some examples like this already
 - A number is zero OR one plus another number
 - A list is empty OR an element cons-ed to another list

“OR” for types

- Pairs gave us “AND”
- What does “OR” look like for types?
 - `(OR Number Boolean)` should be the type of values that are either a number or a boolean
 - Want a tag so we can check which one it is
 - Called a “constructor”
 - not the same as Java/OOP constructor
- Saw some examples like this already
 - A number is zero OR one plus another number
 - A list is empty OR an element cons-ed to another list
- Racket lets us define our own types mixing AND and OR

First example

First example

```
(define-type Shape  
  (Rectangle [length : Number]  
             [width : Number])  
  (Circle [radius : Number]))
```

- Shape is a *datatype*

First example

```
(define-type Shape  
  (Rectangle [length : Number]  
             [width : Number])  
  (Circle [radius : Number]))
```

- Shape is a *datatype*
- It has two *constructors*, Rectangle and Circle

First example

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- Shape is a *datatype*
- It has two *constructors*, Rectangle and Circle
 - i.e. a Shape is a circle OR a rectangle

First example

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- Shape is a *datatype*
- It has two *constructors*, Rectangle and Circle
 - i.e. a Shape is a circle OR a rectangle
- Rectangle has two fields, named length and width, with type Number

First example

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- Shape is a *datatype*
- It has two *constructors*, Rectangle and Circle
 - i.e. a Shape is a circle OR a rectangle
- Rectangle has two fields, named length and width, with type Number
 - length and width are just names we choose

First example

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- Shape is a *datatype*
- It has two *constructors*, Rectangle and Circle
 - i.e. a Shape is a circle OR a rectangle
- Rectangle has two fields, named length and width, with type Number
 - length and width are just names we choose
- Circle has one field named radius with type Number

Constructors

```
(define-type Shape  
  (Rectangle [length : Number]  
             [width : Number])  
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't “do” anything

Constructors

```
(define-type Shape  
  (Rectangle [length : Number]  
             [width : Number])  
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't “do” anything
 - Functional language

Constructors

```
(define-type Shape  
  (Rectangle [length : Number]  
             [width : Number])  
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't "do" anything
 - Functional language
- Gives the primitive way to create a value of that type

Constructors

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't "do" anything
 - Functional language
- Gives the primitive way to create a value of that type
- Can use as functions

Constructors

```
(define-type Shape  
  (Rectangle [length : Number]  
             [width : Number])  
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't "do" anything
 - Functional language
- Gives the primitive way to create a value of that type
- Can use as functions
 - E.g. A rectangle contains two numbers

Constructors

```
(define-type Shape  
  (Rectangle [length : Number]  
             [width : Number])  
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't "do" anything
 - Functional language
- Gives the primitive way to create a value of that type
- Can use as functions
 - E.g. A rectangle contains two numbers
 - Calling `Rectangle` on two numbers returns a value of type `Shape`

Constructors

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't "do" anything
 - Functional language
- Gives the primitive way to create a value of that type
- Can use as functions
 - E.g. A rectangle contains two numbers
 - Calling `Rectangle` on two numbers returns a value of type `Shape`
 - `Rectangle : (Number Number -> Shape)`

Constructors

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't "do" anything
 - Functional language
- Gives the primitive way to create a value of that type
- Can use as functions
 - E.g. A rectangle contains two numbers
 - Calling `Rectangle` on two numbers returns a value of type `Shape`
 - `Rectangle : (Number Number -> Shape)`
 - `Circle : (Number -> Shape)`

Constructors

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))
```

- A constructor for a datatype doesn't "do" anything
 - Functional language
- Gives the primitive way to create a value of that type
- Can use as functions
 - E.g. A rectangle contains two numbers
 - Calling `Rectangle` on two numbers returns a value of type `Shape`
 - `Rectangle : (Number Number -> Shape)`
 - `Circle : (Number -> Shape)`
 - Because it only has 1 field

Creating values of a datatype

Creating values of a datatype

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))

(define tv (Rectangle 16 9))
(define loonie (Circle 1))
```

- A Shape either has two numbers OR one number

Creating values of a datatype

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))

(define tv (Rectangle 16 9))
(define loonie (Circle 1))
```

- A Shape either has two numbers OR one number
 - Flit keeps a tag on each value of type Shape

Creating values of a datatype

```
(define-type Shape
  (Rectangle [length : Number]
             [width : Number])
  (Circle [radius : Number]))

(define tv (Rectangle 16 9))
(define loonie (Circle 1))
```

- A Shape either has two numbers OR one number
 - Flit keeps a tag on each value of type Shape
 - Tells us if it's a Rectangle or Circle

Auto-generated Functions

- Flit makes “checkers” and “getters”

Auto-generated Functions

- Flit makes “checkers” and “getters”
 - We won't use these very much

Auto-generated Functions

- Flit makes “checkers” and “getters”
 - We won't use these very much

Auto-generated Functions

- Flit makes “checkers” and “getters”
 - We won’t use these very much

```
(Rectangle? tv)
(Circle? tv)
(Rectangle-length tv)
;; (test/exn (Circle-radius tv) "")
;; raises an error, no such field present
```

Pattern matching

- Don't want to accidentally get a field that doesn't exist

Pattern matching

- Don't want to accidentally get a field that doesn't exist
- Almost always want to use the fields in the solution

Pattern matching

- Don't want to accidentally get a field that doesn't exist
- Almost always want to use the fields in the solution
- Solution: pattern-matching

Pattern matching

- Don't want to accidentally get a field that doesn't exist
- Almost always want to use the fields in the solution
- Solution: pattern-matching

Pattern matching

- Don't want to accidentally get a field that doesn't exist
- Almost always want to use the fields in the solution
- Solution: pattern-matching

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))])
)
(area tv)
(area loonie)
```

Pattern matching

- Don't want to accidentally get a field that doesn't exist
- Almost always want to use the fields in the solution
- Solution: pattern-matching

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))])
)
(area tv)
(area loonie)
```

144

3.14

Pattern Match Syntax

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:
 - (constructorName variableNames ...)

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:
 - (constructorName variableNames ...)
 - in brackets, even if constructor takes 0 arguments

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:
 - (constructorName variableNames ...)
 - in brackets, even if constructor takes 0 arguments
- Expression after those brackets is the result to produce for that branch

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:
 - (constructorName variableNames ...)
 - in brackets, even if constructor takes 0 arguments
- Expression after those brackets is the result to produce for that branch
- l, w, and r are just the names we choose for the variables

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:
 - (constructorName variableNames ...)
 - in brackets, even if constructor takes 0 arguments
- Expression after those brackets is the result to produce for that branch
- l, w, and r are just the names we choose for the variables
 - If shp is a rectangle, they're assigned the values of its length and width fields

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:
 - (constructorName variableNames ...)
 - in brackets, even if constructor takes 0 arguments
- Expression after those brackets is the result to produce for that branch
- l, w, and r are just the names we choose for the variables
 - If shp is a rectangle, they're assigned the values of its length and width fields
 - Order matters: based on order of fields when type was defined

Pattern Match Syntax

```
(define (area [shp : Shape]) : Number
  (type-case Shape shp
    [(Rectangle l w)
     (* l w)]
    [(Circle r)
     (* 3.14 (* r r))]))
```

- Each branch starts with:
 - (constructorName variableNames ...)
 - in brackets, even if constructor takes 0 arguments
- Expression after those brackets is the result to produce for that branch
- l, w, and r are just the names we choose for the variables
 - If shp is a rectangle, they're assigned the values of its length and width fields
 - Order matters: based on order of fields when type was defined
 - If shp is a circle, then r is gets value of radius field

General Syntax for Pattern Matching

General Syntax for Pattern Matching

```
(define-type DataType
  (ConstructorName1 [field1a : FieldType1a]
    ... [field1z : FieldType1x])
  (ConstructorName2 [field2a : FieldType2a]
    ... [field2z : FieldType2y])
  ...
  (ConstructorNameN [fieldNa : FieldTypeNa]
    ... [field1z : FieldTypeNz]))
```

- Can match on a value of that type as follows:

General Syntax for Pattern Matching

```
(define-type DataType
  (ConstructorName1 [field1a : FieldType1a]
    ... [field1z : FieldType1x])
  (ConstructorName2 [field2a : FieldType2a]
    ... [field2z : FieldType2y])
  ...
  (ConstructorNameN [fieldNa : FieldTypeNa]
    ... [field1z : FieldTypeNz]))
```

- Can match on a value of that type as follows:

General Syntax for Pattern Matching

```
(define-type DataType
  (ConstructorName1 [field1a : FieldType1a]
    ... [field1z : FieldType1x])
  (ConstructorName2 [field2a : FieldType2a]
    ... [field2z : FieldType2y])
  ...
  (ConstructorNameN [fieldNa : FieldTypeNa]
    ... [field1z : FieldTypeNz]))
```

- Can match on a value of that type as follows:

```
(type-case DataType thingToMatchOn
  [(ConstructorName1 var1a var1b ... var1x)
   returnExprForCase1]
  [(ConstructorName2 var2a var2b ... var2y)
   returnExprForCase2]
  ...
  [(ConstructorNameN varNa varNb ... varNz)
   returnExprForCaseN])
```

Semantics of Pattern Matching

- Ctor is short for “constructor”

Semantics of Pattern Matching

- Ctor is short for “constructor”

Definition (Semantics of Pattern Matching)

A pattern match expression

Semantics of Pattern Matching

- Ctor is short for “constructor”

Definition (Semantics of Pattern Matching)

A pattern match expression

Semantics of Pattern Matching

- Ctor is short for “constructor”

Definition (Semantics of Pattern Matching)

A pattern match expression

```
(type-case TYPE (Ctor arg1 ... argn)
  [(OtherCtor1 x1a ... x1z) otherResult1]
  ...
  [(Ctor y1 ... yn) resultExpr]
  ...
  [(OtherCtorM xMa ... xMz) otherResultM])
```

is equal to: $[y1 \Rightarrow arg1] \dots [yn \Rightarrow argn] resultExpr$

- i.e. result with $y1$ replaced by $arg1$ and ... yn replaced by $argn$

- Once again, substitution is the key operation

- Once again, substitution is the key operation
- The value of a type-case is the result of whatever branch matches

- Once again, substitution is the key operation
- The value of a type-case is the result of whatever branch matches
 - with the concrete arguments used as values for the pattern variables $y_1 \dots y_n$ in the matching branch

- Once again, substitution is the key operation
- The value of a type-case is the result of whatever branch matches
 - with the concrete arguments used as values for the pattern variables $y_1 \dots y_n$ in the matching branch
- We don't usually know which constructor is used

- Once again, substitution is the key operation
- The value of a type-case is the result of whatever branch matches
 - with the concrete arguments used as values for the pattern variables $y_1 \dots y_n$ in the matching branch
- We don't usually know which constructor is used
 - e.g. usually we match on a variable or result of function call

- Once again, substitution is the key operation
- The value of a type-case is the result of whatever branch matches
 - with the concrete arguments used as values for the pattern variables $y_1 \dots y_n$ in the matching branch
- We don't usually know which constructor is used
 - e.g. usually we match on a variable or result of function call
 - Way of saying “whatever the value is, this is how we'll handle it”

- Once again, substitution is the key operation
- The value of a type-case is the result of whatever branch matches
 - with the concrete arguments used as values for the pattern variables $y_1 \dots y_n$ in the matching branch
- We don't usually know which constructor is used
 - e.g. usually we match on a variable or result of function call
 - Way of saying “whatever the value is, this is how we'll handle it”
- But, other equations will let us simplify the matched value

Semantics Example

```
(area tv)
= (area (Rectangle 16 9))
= (type-case Shape (Rectangle 16 9)
    [(Rectangle l w) (* l w)]
    [(Circle r) (* 3.14 (* r r))])
= [l => 16][w => 9](* l w)
= (* 16 9)
= 144
```

- Missing a case in pattern matching is a *syntax error*

Total Matching

- Missing a case in pattern matching is a *syntax error*
- Lets us know we are safe

- Missing a case in pattern matching is a *syntax error*
- Lets us know we are safe
 - No “null pointer” or “out of bounds” errors

- Missing a case in pattern matching is a *syntax error*
- Lets us know we are safe
 - No “null pointer” or “out of bounds” errors
- If you write a pattern match, and each branch is correct, then your program is correct

- Missing a case in pattern matching is a *syntax error*
- Lets us know we are safe
 - No “null pointer” or “out of bounds” errors
- If you write a pattern match, and each branch is correct, then your program is correct
- Can add an else clause to handle multiple cases

- Missing a case in pattern matching is a *syntax error*
- Lets us know we are safe
 - No “null pointer” or “out of bounds” errors
- If you write a pattern match, and each branch is correct, then your program is correct
- Can add an else clause to handle multiple cases
 - See Racket window

Most Languages are Unsafe

- E.g. Imagine a C++ struct for a library

Most Languages are Unsafe

- E.g. Imagine a C++ struct for a library
 - You might remember this from CS 115

Most Languages are Unsafe

- E.g. Imagine a C++ struct for a library
 - You might remember this from CS 115

Most Languages are Unsafe

- E.g. Imagine a C++ struct for a library
 - You might remember this from CS 115

```
enum ItemType {  
    Book,  
    DVD  
}  
  
struct ItemInfo {  
    string title;  
    ItemType itemType;  
    // Only needed for books  
    string author;  
    // Only needed for DVDs  
    int runningTime;  
}
```

- No way to enforce that we don't have running time for book

Make Illegal States Unrepresentable

Alternative with Datatypes

- Instead, Flit can do:

Alternative with Datatypes

- Instead, Flit can do:

Alternative with Datatypes

- Instead, Flit can do:

```
(define-type ItemInfo
  (Book [title : String] [author : String])
  (DVD [title : String] [runningTime : Number]))

(define (print-info item)
  (type-case ItemInfo item
    [(Book title author)
     (string-append title
                     (string-append ", by " author)))]
    [(Book title runningTime)
     (string-append title
                     (string-append ": runtime "
                                     (to-string runningTime))))]))
```

- No way to give a DVD an author or Book a running time

Alternative with Datatypes

- Instead, Flit can do:

```
(define-type ItemInfo
  (Book [title : String] [author : String])
  (DVD [title : String] [runningTime : Number]))

(define (print-info item)
  (type-case ItemInfo item
    [(Book title author)
     (string-append title
       (string-append ", by " author)))]
    [(Book title runningTime)
     (string-append title
       (string-append ": runtime "
         (to-string runningTime))))]))
```

- No way to give a DVD an author or Book a running time
- Can't accidentally e.g. print the author of a DVD, because there won't even be an author in scope

Application: Representing Failure

- In flit standard library

Application: Representing Failure

- In flit standard library

Application: Representing Failure

- In flit standard library

```
(define-type (Optionof 'a)  
  (none)  
  (some [v : 'a]))
```

- Alternative to using NULL to mark a value as absent

Application: Representing Failure

- In flit standard library

```
(define-type (Optionof 'a)  
  (none)  
  (some [v : 'a]))
```

- Alternative to using NULL to mark a value as absent
- Pattern matching guarantees no null pointer errors

Application: Representing Failure

- In flit standard library

```
(define-type (Optionof 'a)  
  (none)  
  (some [v : 'a]))
```

- Alternative to using NULL to mark a value as absent
- Pattern matching guarantees no null pointer errors
 - We'll see a more detailed example

Application: Representing Failure

- In flit standard library

```
(define-type (Optionof 'a)  
  (none)  
  (some [v : 'a]))
```

- Alternative to using NULL to mark a value as absent
- Pattern matching guarantees no null pointer errors
 - We'll see a more detailed example
- Type system expresses whether can be empty or not

Application: Representing Failure

- In flit standard library

```
(define-type (Optionof 'a)  
  (none)  
  (some [v : 'a]))
```

- Alternative to using NULL to mark a value as absent
- Pattern matching guarantees no null pointer errors
 - We'll see a more detailed example
- Type system expresses whether can be empty or not
 - If it's not `Optionof`, then value can't be missing

Example: Safe Head

Example: Safe Head

```
(define (safeHead [l : (Listof 'a)]) : (Optionof 'a)
  (type-case (Listof 'a) l
    [(empty)
     (none)]
    [(cons h t)
     (some h)]))
(safeHead '())
(safeHead (list 1 2 3 4 ))
```

Example: Safe Head

```
(define (safeHead [l : (Listof 'a)]) : (Optionof 'a)
  (type-case (Listof 'a) l
    [(empty)
     (none)]
    [(cons h t)
     (some h)]))
(safeHead '())
(safeHead (list 1 2 3 4 ))
```

```
(none)
(some 1)
```

Example: Optional Arguments

- Option gives a way to say “use a default value for this argument”

Example: Optional Arguments

- Option gives a way to say “use a default value for this argument”

Example: Optional Arguments

- Option gives a way to say “use a default value for this argument”

```
(define (range [start : Number] [end : Number]
               [maybeGap : (Optionof Number)])
  : (Listof Number)
  ;; Default gap of 1 if nothing is given
  (let* ([gap (type-case (Optionof Number) maybeGap
                        [(none) 1]
                        [(some x) x])])
    (if (<= end start)
        ;; Base case, nothing in range if end is before start
        '()
        ;; Recursive case: include the start
        ;; then add that to the range starting at the next number
        (cons start (range (+ gap start) end maybeGap))
        )))

(range 0 10 (none))
(range 0 100 (some 5))
```

Example: Association Lists

- Can use lists of pairs as a key-value store

Example: Association Lists

- Can use lists of pairs as a key-value store
- Want to search for element in the list whose key is present

Example: Association Lists

- Can use lists of pairs as a key-value store
- Want to search for element in the list whose key is present
 - What if it's not in the list?

Example: Association Lists

- Can use lists of pairs as a key-value store
- Want to search for element in the list whose key is present
 - What if it's not in the list?

Example: Association Lists

- Can use lists of pairs as a key-value store
- Want to search for element in the list whose key is present
 - What if it's not in the list?

```
(define (lookup [key : Number]
               [pairs : (Listof (Number * 'a))])
  : (Optionof 'a)
  (type-case (Listof (Number * 'a)) pairs
    [empty ;; If list is empty, then the key can't be present
     (none)]
    [(cons h t)
     (if (= key (fst h)) ;; If the key matches, we're done
         (some (snd h))
         (lookup key t))]))

(lookup 2 (list (pair 1 "one") (pair 2 "two") (pair 4 "four")))
(lookup 3 (list (pair 1 "one") (pair 2 "two") (pair 4 "four")))
```

Example: Association Lists

- Can use lists of pairs as a key-value store
- Want to search for element in the list whose key is present
 - What if it's not in the list?

```
(define (lookup [key : Number]
                [pairs : (Listof (Number * 'a))])
  : (Optionof 'a)
  (type-case (Listof (Number * 'a)) pairs
    [empty ;; If list is empty, then the key can't be present
     (none)]
    [(cons h t)
     (if (= key (fst h)) ;; If the key matches, we're done
         (some (snd h))
         (lookup key t))]))

(lookup 2 (list (pair 1 "one") (pair 2 "two") (pair 4 "four")))
(lookup 3 (list (pair 1 "one") (pair 2 "two") (pair 4 "four")))
```

```
(some "two")
(none)
```

Options in the Real World

- Swift (e.g. writing iPhone apps)

Options in the Real World

- Swift (e.g. writing iPhone apps)

Options in the Real World

- Swift (e.g. writing iPhone apps)

```
enum Optional<Wrapped> {  
    case none  
    case some(Wrapped)  
}
```

- Rust (Safer alternative to C/C++)

Options in the Real World

- Swift (e.g. writing iPhone apps)

```
enum Optional<Wrapped> {  
    case none  
    case some(Wrapped)  
}
```

- Rust (Safer alternative to C/C++)

Options in the Real World

- Swift (e.g. writing iPhone apps)

```
enum Optional<Wrapped> {  
    case none  
    case some(Wrapped)  
}
```

- Rust (Safer alternative to C/C++)

```
pub enum Option<T> {  
    None,  
    Some(T),  
}
```

Datatype to End All Datatypes

Datatype to End All Datatypes

```
(define-type (Either 'a 'b)
  (Left [inLeft : 'a])
  (Right [inRight : 'b])
)
```

- Can define all (non-recursive) datatypes with this and pairs

Datatype to End All Datatypes

```
(define-type (Either 'a 'b)  
  (Left [inLeft : 'a])  
  (Right [inRight : 'b])  
)
```

- Can define all (non-recursive) datatypes with this and pairs
- e.g. Shape is like (Either (Number * Number) Number)

Either Example

Either Example

```
;; Like lookup, but also errors if key occurs multiple times  
;; Either lets us encode a message in the failure case  
(define (lookupUnique [key : Number]  
  [pairs : (Listof (Number * 'a))])  
  : (Either String 'a)  
  (type-case (Listof (Number * 'a)) pairs  
    [empty ;; If list is empty, then the key can't be present  
      (Left "Key not found!")]  
    [(cons h t)  
      (let* ([rec (lookupUnique key t)])  
        (if (= key (fst h)) ;; Check for dupes  
          (type-case (Either String 'a) rec  
            ;; If head has key but tail doesn't, success  
            [(Left msg) (Right (snd h))]  
            [(Right val) (Left "Duplicate keys in list")])  
          ;; If head didn't have key  
          ;; Then success/failure is what tail found  
          rec)])))
```